CareerX: A RAG Framework for Personalized AI-Driven Career Guidance

A Project Report

submitted by

SARHAN QADIR KVM	**EKC22CS051**
FATHIMA HANA K	**EKC22CS016**
RIFA	**EKC22CS049**
MOHAMMED SINAN T	**EKC22CS035**

to

the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

of

Bachelor of Technology

in

Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ERANAD KNOWLEDGE CITY TECHNICAL CAMPUS,

MANJERI

April-2026

DECLARATION

We Sarhan Qadir KVM, Fathima Hana k, Rifa and Mohammed Sinan T hereby declare that the project report **CareerX: A RAG Framework for Personalized AI-Driven Career Guidance**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of Ms. Jasira KT

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Manjeri
13-4-2026

Sarhan Qadir KVM
Fathima Hana k
Rifa
Mohammed Sinan T

DEPT. OF COMPUTER SCIENCE ENGINEERING

ERANAD KNOWLEDGE CITY TECHNICAL CAMPUS, MANJERI

2025 - 26

CERTIFICATE

This is to certify that the report entitled **CareerX: A RAG Framework for Personalized AI-Driven Career Guidance** submitted by **Sarhan Qadir KVM** (EKC22CS051), **Fathima Hana KP** (EKC22CS016), **Rifa** (EKC22CS049) and **Mohammed Sinan T** (EKC22CS035), to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by them under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide	**Project Coordinator**	**Head of Department**
MS JASIRA KT	Ms. Anu K S	Ms. Anu K S
Assistant Professor	Professor and Head	Professor and Head
Dept.of CSE	Dept.of CSE	Dept.of CSE
EKCTC	EKCTC	EKCTC
Manjeri	Manjeri	Manjeri

Acknowledgement

We take this opportunity to express our deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to our Principal **Dr. Adarsh T K** and **Ms. Anu K S**, Head of Department and Project Coordinator, Computer Science and Engineering, Eranad Knowledge City Technical Campus, Manjeri for providing us with all the necessary facilities and support.

We would like to place on record our sincere gratitude to our project guide **MS.JASIRA KT**, Assistant Professor, Computer Science and Engineering, Eranad Knowledge City Technical Campus for the guidance and mentorship throughout the course.

Finally we thank our family, friends and all other Professors who contributed to the succesful fulfilment of this project work.

Sarhan Qadir KVM

Fathima Hana K

Rifa

Mohammed Sinan T

Abstract

Career guidance systems must adapt to rapidly shifting labor markets, yet traditional platforms rely on static databases while standalone large language models (LLMs) are limited by training data cutoffs and prone to hallucination. This project presents **CareerX**, a web-based career guidance system that augments LLM generation with live web retrieval through a SearXNG-based meta-search pipeline.

The system dynamically generates personalized intake questionnaires via LLM-driven schema generation, formulates targeted search queries, scrapes and cleans live web sources, and synthesizes structured career dashboards through Zod-enforced schema-constrained generation. Unlike canonical Retrieval-Augmented Generation (RAG) architectures that retrieve from pre-built vector stores, CareerX performs direct web retrieval — live search results are scraped and injected into the LLM context window, trading embedding-based semantic precision for real-time data currency.

The system produces location-specific salary data, source-attributed university recommendations, and traceable URL citations that standalone LLMs cannot provide without retrieval augmentation. Evaluation across five simulated user profiles demonstrates a mean end-to-end pipeline time of 66.7 seconds (SD = 6.2s), with an average of 37.7 web sources retrieved per session and 79% schema completeness across generated outputs. CareerX demonstrates the practical viability of combining real-time web retrieval with schema-constrained generative AI for domain-specific advisory applications.

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ATS	Applicant Tracking System
API	Application Programming Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
JS	JavaScript
LLM	Large Language Model
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit
UI	User Interface
URL	Uniform Resource Locator
WAG	Web-Augmented Generation

Chapter 1

Introduction

In the modern professional landscape, technological disruptions are reshaping the nature of work at an unprecedented pace. Fields that were once stable for decades are now transforming within years. Artificial Intelligence, Machine Learning, Cloud Computing, and Data Science have fundamentally altered the skill requirements of nearly every industry. Career guidance systems that were adequate a decade ago are no longer sufficient to serve today's students and professionals navigating this volatile labor market.

Conventional career guidance systems rely on static, manually maintained databases such as O*NET [1], compiled by the United States Department of Labor, which catalog occupational information including tasks, skills, knowledge areas, and work activities across hundreds of professions. While comprehensive, these databases require extensive human effort to maintain and are inherently reactive — they reflect the state of the labor market at the time of their last update, not its current state. In fields such as Generative AI, Prompt Engineering, and MLOps, entire job categories can emerge and reach high demand within months, well before any static database can record them.

When standalone large language models (LLMs) are used for career counseling, they face two fundamental limitations: training data cutoffs that prevent awareness of current market conditions, and hallucination of fabricated institutions, salary figures, or career pathways [8]. A student who receives career advice from a standalone LLM may be directed toward a university program that has since closed, a salary expectation that no longer matches the market, or a certification that has been superseded by a newer standard. These errors are particularly dangerous because the LLM delivers

its responses with high confidence and fluency, offering no signal to the user that the information may be unreliable.

1.1 Problem Statement

The central problem this project addresses is the disconnect between the pace of change in the labor market and the responsiveness of career guidance systems. Current digital advising tools exhibit three key failure modes.

Static Data Problem: Most career guidance platforms are built on periodically updated databases that lag behind actual market conditions by months or years. For rapidly evolving fields, this lag makes their advice potentially misleading. A job title such as "AI Prompt Engineer" or "MLOps Specialist" may not exist in a database compiled two years ago, even though thousands of such positions are currently being advertised.

LLM Hallucination Problem: Standalone large language models, when used for career guidance, are prone to confidently generating fabricated information. Ji et al. [8] document this extensively, classifying hallucinations into intrinsic (contradicting source material) and extrinsic (unverifiable against any source) types. In career guidance, these errors carry real consequences: a student may spend years pursuing an academic path based on hallucinated advice.

Rigid Questionnaire Problem: Existing digital career platforms typically implement tree-based or static questionnaires that lead to predefined outcome categories. These questionnaires cannot adapt mid-session to a user's unique context. A student interested in artificial intelligence who also has strong entrepreneurial ambitions may receive a generic recommendation because the questionnaire was not designed to handle the intersection of these interests.

Recent systems such as those described by Jawhar et al. [20] have explored AI-powered university and career guidance using LLMs, but rely primarily on static knowledge bases without any mechanism for real-time market data integration.

1.2 Motivation

The motivation behind this project is to bridge the gap between static counseling and live industry intelligence. By augmenting LLM generation with real-time web retrieval, a system can ground its recommendations in current, verifiable data rather than relying solely on potentially outdated training knowledge.

This motivation is particularly strong in the Indian context. India's higher education system produces over 1.5 million engineering graduates annually, many of whom face significant career uncertainty. The mismatch between academic preparation and industry expectations is well-documented: graduates frequently lack awareness of current hiring criteria, compensation benchmarks, and emerging specializations. A career guidance system that retrieves live data from job portals, Reddit communities, LinkedIn discussions, and university websites can provide substantially more relevant and actionable advice than one relying on a static knowledge base.

Moreover, the geographic dimension of career guidance is critical but often overlooked. Salary expectations for a software engineer in Tier-1 cities in India differ substantially from those in the Middle East, the United Kingdom, or the United States. A retrieval-augmented system that queries location-specific salary surveys and job listings can provide advice calibrated to the user's specific geographic context.

1.3 Objectives

The primary objectives of the CareerX project are:

- To develop a dynamic questionnaire generation mechanism that adapts intake questions to user context through LLM-driven schema generation, moving beyond rigid static questionnaire designs.
- To build a web-augmented retrieval pipeline that fetches, scrapes, and processes live web sources through SearXNG meta-search aggregation, grounding all recommendations in current, publicly available data.
- To implement a schema-constrained synthesis layer using Zod validation to enforce structured, predictable JSON outputs suitable for direct UI rendering.

- To generate location-aware career dashboards including career pathways with match percentages, salary ranges in local currency, ranked university recommendations, job listings from region-appropriate portals, skill gap roadmaps, and source citations.
- To evaluate the system's performance across diverse user profiles through quantitative metrics including response time, source retrieval volume, and output completeness.
- To implement a privacy-preserving ephemeral architecture that processes all user data within the active browser session without requiring authentication or persistent server-side storage.

1.4 System Overview

CareerX is an end-to-end career guidance system that combines adaptive user profiling with real-time web data retrieval and schema-constrained LLM synthesis. The system operates through a multi-stage pipeline that progressively transforms a user's role, location, and personal context into a comprehensive, source-attributed career dashboard.

The pipeline consists of five sequential stages: (1) minimal intake via a web form, (2) LLM-generated personalized questionnaire, (3) targeted web search queries submitted to SearXNG, (4) scraping and assembly of a context payload of approximately 80,000–100,000 characters, and (5) Zod-validated dashboard synthesis by the LLM. The resulting dashboard includes career pathways, salary ranges, university recommendations, job listings, skill gap roadmaps, and source citations that allow users to independently verify recommendations.

1.5 Scope and Significance

CareerX addresses a critical gap: no existing open system simultaneously combines real-time web retrieval, adaptive multi-stage user profiling, and schema-constrained output for the career guidance domain.

Commercial tools such as Perplexity.ai address real-time retrieval but operate on single-turn queries and produce unstructured text rather than domain-specific structured career dashboards. Traditional platforms such as O*NET provide domain specificity but lack live data and adaptive intake. Canonical RAG architectures retrieve from pre-built vector stores that require periodic re-indexing and cannot provide real-time data currency.

The significance of CareerX lies in demonstrating that a practical, privacy-preserving, real-time career guidance system can be built using publicly available open-source tools — a self-hosted SearXNG meta-search engine, the Google Gemini API, and Next.js — without requiring proprietary data infrastructure.

1.6 Report Organization

The remainder of this report is organized as follows. Chapter 2 reviews related work in career guidance systems, LLMs, and retrieval-augmented generation. Chapter 3 describes the system requirements and methodology. Chapter 4 presents the system architecture and design. Chapter 5 covers implementation details. Chapter 6 presents evaluation results and analysis. Chapter 7 concludes with a summary of contributions and future work directions.

Chapter 2

Literature Survey

This chapter reviews existing work in five areas relevant to CareerX: traditional career guidance systems, large language models in advisory contexts, retrieval-augmented generation, schema-constrained LLM output, and job recommendation and skill gap analysis systems.

2.1 Traditional Career Guidance Systems

Career guidance has historically relied on expert systems and static occupational databases. The O*NET framework [1], maintained by the United States Department of Labor, provides the most widely used occupational database in the world, cataloging over 900 occupational profiles. While comprehensive, O*NET is manually curated and updated approximately annually, causing it to lag consistently behind fast-moving sectors such as artificial intelligence and blockchain.

Liu et al. [1] proposed a framework for intelligent career recommendation for undergraduates using structured matching between student profiles and career options over a curated knowledge base. Their system demonstrated that structured intake profiling combined with knowledge-base matching improves recommendation relevance, but the knowledge base required manual curation and could not generalize to emerging fields.

Petkov [3] studied the modernization of youth career services through AI and digital media literacy, identifying institutional barriers to modernization and the need for deployable, low-infrastructure career guidance tools — a design constraint that directly informed CareerX's architecture.

Jawhar et al. [20] introduced an AI-powered university and career guidance system for high school students that submits detailed student profiles to the OpenAI GPT-4 API for personalized recommendations. This work represents the closest prior system to CareerX in terms of user interaction model, demonstrating the viability of LLM-driven intake and recommendation for educational guidance. However, it relies entirely on GPT-4's training data without any real-time retrieval, preventing it from providing current salary data, recently launched programs, or live job listings.

2.2 Large Language Models in Guidance

Brown et al. [4] demonstrated that large-scale autoregressive language models exhibit broad reasoning and few-shot learning capabilities, enabling them to perform advisory tasks with minimal task-specific training. This established the foundation for prompt-based advisory systems where user context is provided directly in the prompt.

However, Ji et al. [8] provide a comprehensive survey of hallucination in natural language generation, documenting how LLMs frequently generate plausible but fabricated information. They classify hallucinations into intrinsic (contradicting source material) and extrinsic (unverifiable against any source) types. In career guidance, extrinsic hallucination is the dominant risk: an LLM may generate a fabricated university program, an outdated salary figure, or a nonexistent certification pathway. Users who cannot distinguish hallucinated from verified information may make consequential life decisions based on confabulated guidance.

Neumann et al. [5] examined the adoption of ChatGPT in academic settings, warning against its use for time-sensitive or domain-specific information without appropriate grounding mechanisms. This finding directly motivates CareerX's retrieval augmentation approach.

2.3 Retrieval-Augmented Generation

Lewis et al. [6] introduced Retrieval-Augmented Generation (RAG), demonstrating that grounding LLM outputs in retrieved documents significantly reduces hallucination and improves factual accuracy for knowledge-intensive tasks. The canonical RAG

architecture combines dense retrieval from a vector store with a sequence-to-sequence generator, achieving substantial improvements over standalone language models on open-domain QA benchmarks.

Asai et al. [9] introduced Self-RAG, where the model learns to decide when retrieval is necessary and critiques its own outputs using reflection tokens, improving both efficiency and quality over always-retrieve architectures. Jiang et al. [10] proposed FLARE (Forward-Looking Active Retrieval), which anticipates future information needs during generation by triggering retrieval when the model generates low-confidence tokens.

Gupta et al. [12] survey RAG evolution, categorizing approaches into naive, advanced, and modular RAG architectures. They highlight that real-time web retrieval — as employed in CareerX — represents a distinct variant termed "web-augmented generation" that prioritizes data currency over semantic precision.

Commercial systems such as Perplexity.ai demonstrate the viability of real-time web retrieval augmented generation at scale. However, these are general-purpose and produce unstructured textual answers rather than domain-specific structured outputs optimized for UI rendering. CareerX differs by combining live web retrieval with schema-constrained synthesis specifically designed for the career guidance domain.

2.4 RAG in Educational Applications

Siriwardhana et al. [14] conducted a systematic survey of RAG applications in educational technology, documenting that RAG approaches consistently outperform standalone LLMs in accuracy and relevance across educational domains. The survey identifies key success factors: transparency about which sources were used, structured output formats, and domain restriction that prevents the system from generating confident responses outside its knowledge scope. CareerX incorporates all three through source citations, structured JSON output, and Zod schema validation.

Neumann et al. [19] surveyed RAG-based chatbot deployments in higher education, finding that grounding responses in retrieved materials significantly improved student trust and answer accuracy. These findings support the use of retrieval augmentation in the career guidance context.

2.5 Schema-Constrained LLM Output

Geng et al. [13] introduced JSONSchemaBench, demonstrating that without explicit schema enforcement, even capable models frequently produce outputs that fail schema validation, particularly for complex nested structures. Schema-constrained generation substantially improves reliability.

Liu et al. [18] found that users of professional AI systems strongly prefer outputs conforming to predictable schemas, as this enables reliable integration with external systems and reduces cognitive load. This supports the Zod-based validation architecture in CareerX.

2.6 Job Recommendation and Skill Gap Analysis

Du et al. [15] enhanced job recommendation through LLM-based Generative Adversarial Networks, demonstrating that LLMs can improve resume-job matching by generating enriched candidate representations. Mirajkar et al. [17] proposed an AI-driven skill recommendation and resume analysis framework combining NLP-based skill extraction with recommendation logic, validating the practical feasibility of AI-driven skill gap analysis. Kishore and Sreerala [16] demonstrated that real-time job board data integration significantly outperforms historical data for recommendation relevance, providing direct empirical support for CareerX's live retrieval approach.

2.7 Summary

The reviewed literature reveals a consistent trajectory toward more dynamic, retrieval-augmented approaches for career guidance. Table 2.1 summarizes the key differences between CareerX and related systems.

Table 2.1: Comparison of CareerX with Related Work

System	Live Data	Dynamic Q	Schema Output	Citations
O*NET [1]	No	No	Yes	No
Jawhar et al. [20]	No	No	No	No
Perplexity.ai	Yes	No	Partial	Yes
Self-RAG [9]	Partial	No	No	No
CareerX (Ours)	**Yes**	**Yes**	**Yes**	**Yes**

Chapter 3

System Requirement Study and Methodology

3.1 Existing System Limitations

In traditional career guidance, students and professionals depend on human counselors, psychometric instruments, and rule-based web platforms. A critical analysis of existing digital systems reveals the following limitations that motivated the CareerX architecture:

- **Static Data:** Databases like O*NET require manual updates and quickly become outdated in fast-moving sectors. Salary data, certification requirements, and job title terminology change faster than database update cycles.
- **Hallucination Risk:** Standalone LLMs generate plausible but potentially fabricated salary figures, institutions, and career pathways without any mechanism for the user to verify the claims.
- **No Personalization at Intake:** Fixed questionnaires cannot adapt to user context mid-session. A system that asks every user the same five questions cannot account for the intersection of interests, constraints, and goals that characterize real career decisions.
- **Unstructured Output:** LLM outputs without schema enforcement lack predictable structure, making reliable UI integration difficult and resulting in inconsistent user experiences across queries.
- **No Source Traceability:** Users cannot verify the origin of recommendations.

A salary figure without a source URL cannot be trusted in the same way as one linked to a named salary survey.

- **Privacy Costs:** Most recommendation systems require account creation and persistent storage of user data, creating privacy risks and regulatory compliance burdens.

3.2 Proposed System

CareerX addresses these limitations through a five-stage web-augmented retrieval pipeline that operates entirely within a single browser session.

3.2.1 Design Philosophy

The design of CareerX rests on three principles. First, **Currency Over Precision**: CareerX prioritizes accessing the most current publicly available information over the semantic precision of a curated knowledge base. Live web data reflects the labor market as it exists today rather than as it was catalogued years ago. Second, **Schema Over Flexibility**: CareerX prioritizes structured, predictable output over free-form generation. A user who receives a consistently structured career dashboard can navigate and verify its contents more effectively. Third, **Ephemerality Over Persistence**: User data never leaves the active browser session, eliminating authentication needs and reducing data breach risk.

3.2.2 Five-Stage Pipeline

1. **Adaptive Intake:** The user provides name, role, and location through a minimal intake form. A geolocation API supplements the manually provided location for precise geographic salary differentiation.

2. **Dynamic Questionnaire Generation:** The LLM generates a personalized JSON schema defining 5–7 context-specific questions with appropriate input types (multi-select, slider, rating, text, dropdown). The client renders the questionnaire dynamically from this schema.

3. **Query Planning and Web Retrieval:** Based on the completed questionnaire, the LLM formulates 4–8 targeted search queries. Each is submitted to the self-hosted SearXNG instance, retrieving up to 5 pages per query with inter-page delays to avoid rate limiting.

4. **Context Assembly:** The top 10 URLs are scraped for full page content (up to 10,000 characters each after HTML/CSS/script removal). An additional 50 results contribute snippet-level context. Total context: approximately 80,000–100,000 characters.

5. **Schema-Constrained Synthesis:** The assembled context and user profile are submitted to the synthesis LLM. The Vercel AI SDK's `generateObject()` function enforces Zod schema compliance at the generation level, automatically retrying if output fails validation.

3.3 System Requirements

3.3.1 Hardware Requirements

- Processor: Apple Silicon M1 or Intel Core i5 or higher
- RAM: Minimum 16 GB (for running the local SearXNG Docker instance alongside the Next.js development server)
- Storage: 10 GB available (for Docker images and project files)
- Internet Connection: Broadband (minimum 20 Mbps recommended for web scraping)

3.3.2 Software Requirements

- Operating System: macOS 12+, Ubuntu 22.04 LTS, or Windows 11 with WSL2
- Node.js: Version 18 or above (Next.js 15 requirement); Node.js 20 LTS recommended
- Docker: For running the self-hosted SearXNG instance

- Google Gemini API Key: For LLM access (free tier or paid)
- Package Manager: npm or yarn
- Code Editor: Visual Studio Code (recommended)

3.4 Retrieval Methodology

3.4.1 SearXNG Meta-Search Integration

SearXNG is a self-hosted, privacy-respecting meta-search engine aggregating results from Google, Bing, DuckDuckGo, and others. CareerX uses a Dockerized SearXNG instance for three reasons: to avoid individual search provider API rate limits and costs, to aggregate diverse result sets for broader coverage, and to maintain user privacy with no query logging to third parties.

The system generates 4–8 targeted queries per session covering: role-specific salary ranges in the user's location, community opinions from Reddit and LinkedIn (using site-specific query operators), relevant university programs and degree requirements, certification paths and online learning resources, and current job listings on regionally appropriate portals.

3.4.2 Web Scraping Pipeline

A custom web scraper processes retrieved URLs with the following characteristics:

- 12-second timeout per page with graceful AbortController handling
- Graceful HTTP 403/404 degradation (skip and continue to next URL)
- HTML tag, script, and stylesheet removal for clean text extraction
- 10,000-character limit per page to manage LLM context window
- Randomized User-Agent headers to reduce blocking probability
- Banned domain list filtering (dictionary sites, irrelevant aggregators)

3.4.3 Schema-Constrained Synthesis

The synthesis prompt assembles the user questionnaire responses, scraped full-page content from top 10 URLs, and snippet-level context from 50 additional results. The LLM is instructed to generate output conforming to a Zod schema with the following fields:

Table 3.1: CareerX Output Schema Fields

Field	Type	Required
profileSummary	String	Yes
careerPathways	Array (with match %, salary)	Yes
education	Array (degrees + certifications)	Yes
topUniversities	Array (with fees, programs)	Yes
jobs	Array (with portal deep-links)	Yes
skillGaps	Object (year-by-year roadmap)	Yes
startupResources	Array	Optional
scholarships	Array	Optional
professionalBodies	Array	Optional
socialReviews	Array (with source URLs)	Optional
sources	Array (cited URLs)	Optional

3.5 Schema Completeness Metric

Schema completeness is formally defined as:

$$Completeness = \frac{N_{populated}}{N_{total}} \times 100 \tag{3.1}$$

where $N_{total} = 11$ (6 required + 5 optional fields). A field is considered populated if:

- For strings: the trimmed value is non-empty
- For arrays: the length is greater than zero
- For objects: at least one child property is populated

Chapter 4

System Architecture and Design

4.1 System Architecture

CareerX is built on a three-layer architecture that separates concerns across the client interface, AI orchestration, and data retrieval functions. This separation allows each layer to be developed, tested, and optimized independently.

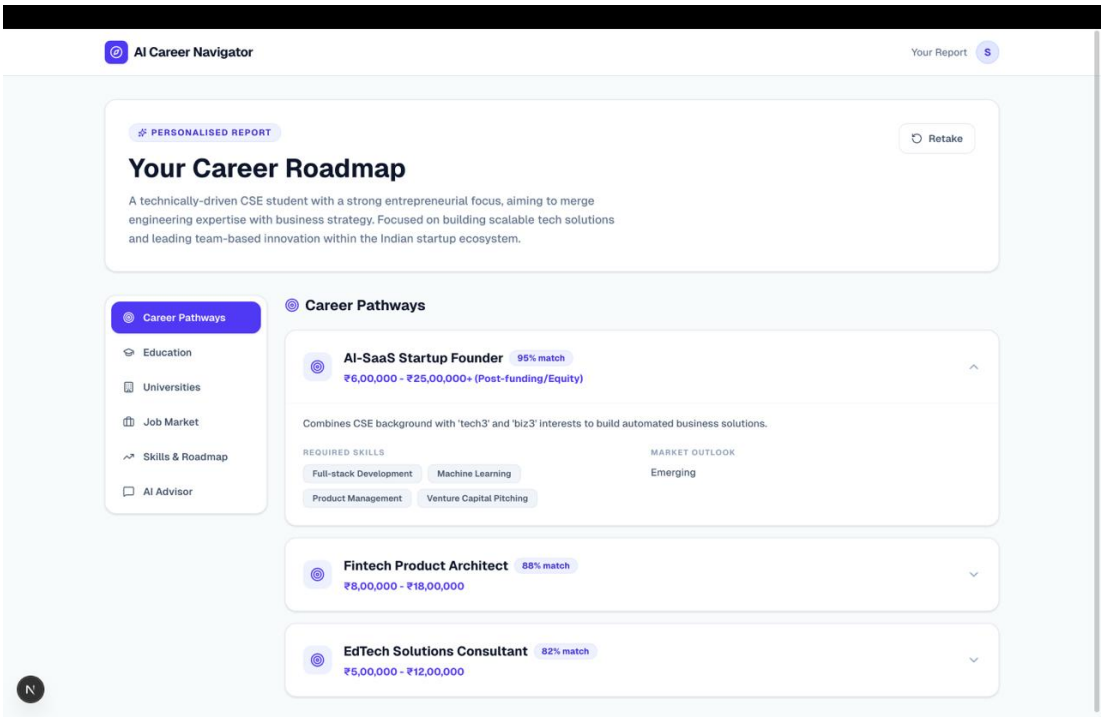

Figure 4.1: CareerX System Architecture: Three-Layer Pipeline

4.1.1 Client Interface Layer

Built on Next.js 15 (App Router) with TailwindCSS and Framer Motion, the client interface renders all UI components dynamically from AI-generated JSON schemas rather than hardcoded component trees. It supports multiple input types — multi-select, sliders, rating scales, and text fields — for questionnaire rendering. The dashboard visualizes career pathways as interactive nodes with match percentages and salary ranges, expandable to show required skills and market demand. Source citation links connect every recommendation to a retrieved web page. All state is managed using React component state and sessionStorage with no server-side persistence.

4.1.2 AI Orchestration Layer

Powered by Google Gemini 1.5 Flash via the Vercel AI SDK, this layer performs three sequential generation tasks:

1. **Questionnaire Schema Generation:** Given the user's role and location, generates a JSON schema defining 5–7 adaptive questions with appropriate input types. Average latency: 2–3 seconds.

2. **Search Query Planning:** Given the completed questionnaire responses, generates 4–8 targeted search queries covering salary data, community opinions, universities, certifications, and job listings. Average latency: 1–2 seconds.

3. **Dashboard Synthesis:** Given the assembled web context and user profile, generates the complete career dashboard JSON conforming to the Zod schema. Average latency: 15–20 seconds.

All LLM outputs are validated against Zod schemas using `generateObject()` to enforce structural correctness before the response reaches the client.

4.1.3 Data Retrieval Engine

A self-hosted SearXNG meta-search instance aggregates results across multiple search engines. The system generates 4–8 targeted search queries per user session, retrieves

up to 5 pages per query (with 1.2-second inter-page delays), and deduplicates results by URL. A custom web scraper with 12-second timeout handling and graceful degradation on inaccessible sources extracts clean text content from the top 10 results. Each scraped page is limited to 10,000 characters after HTML tag, script, and stylesheet removal. A banned domain list filters out dictionary and definition sites that do not contribute career-relevant content.

4.2 Pipeline Data Flow

The complete data flow through the CareerX pipeline is as follows:

1. User submits name, role, and location via the intake form
2. Orchestration layer calls Gemini to generate a questionnaire JSON schema
3. Client renders the questionnaire dynamically; the user completes and submits it
4. Orchestration layer calls Gemini to generate 4–8 targeted search queries
5. Each query is submitted to SearXNG; results are paginated and deduplicated
6. Top 10 URLs are scraped for full content; 50 additional results contribute snippets
7. Assembled context (~80,000–100,000 characters) plus user profile are sent to the synthesis LLM
8. Gemini generates a Zod-validated career dashboard JSON
9. Client renders the dashboard with interactive pathway nodes and source citations

4.3 Functional Modules

4.3.1 Dynamic Questionnaire Module

This module uses LLM-driven schema generation to produce context-sensitive intake questions that adapt to the user's stated role. Unlike static questionnaire systems, the questions change based on user context:

- A **software engineering** role generates questions about programming languages, cloud platform experience, team size preferences, and research versus product orientation.
- A **healthcare** role generates questions about clinical specialization, regulatory context, preferred healthcare setting, and geographic practice preference.
- An **entrepreneurship** role generates questions about funding stage, market focus, team composition, and risk tolerance.

The same rendering component handles all role types without any conditional frontend logic.

4.3.2 Query Planning Module

The query planning module generates search queries that collectively cover all dimensions of the career guidance problem for the user's specific role and location:

- Role-specific salary ranges from annual salary surveys (Glassdoor, AmbitionBox, Levels.fyi)
- Qualitative community opinions from Reddit career communities
- University programs from institutional websites and QS rankings
- Certification paths from credentialing bodies (AWS, Google Cloud, SHRM, etc.)
- Current job listings from region-appropriate portals (LinkedIn, Naukri, Indeed, Bayt)

4.3.3 Recommendation Engine

The synthesis LLM acts as the recommendation engine, synthesizing the assembled web context and user profile into structured career recommendations. The match percentages for career pathways are computed by the LLM based on its holistic assessment of alignment between the user's stated skills, education level, location constraints, and preferences, and the requirements described in the retrieved market data.

4.4 Privacy Architecture

CareerX implements a stateless privacy model:

- All user data is processed within the active browser session using React component state and sessionStorage
- No user data is written to server-side storage at any point in the pipeline
- sessionStorage is automatically cleared when the browser tab is closed
- No authentication or user accounts are required
- The self-hosted SearXNG instance prevents queries from being logged to third-party search providers

This architecture trades persistent user profiles for zero-storage privacy compliance, making CareerX suitable for deployment without data processing agreements.

Figures 4.2 and 4.3 show the CareerX user interface at the role selection and report generation stages.

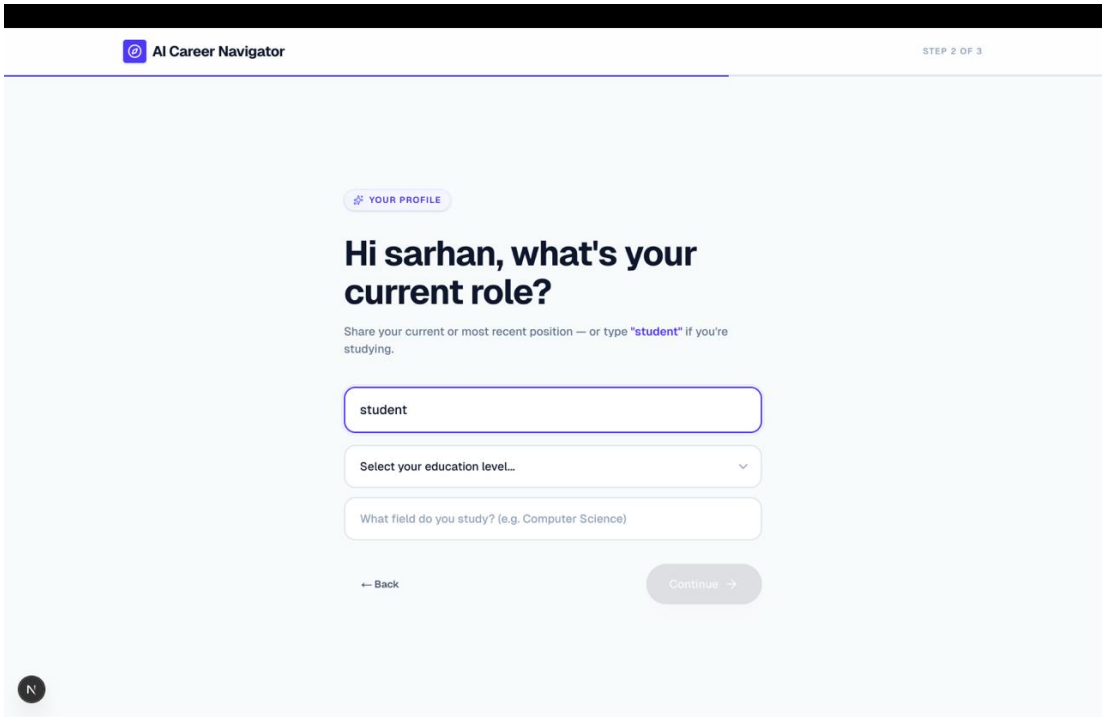

Figure 4.2: CareerX Role Selection Interface

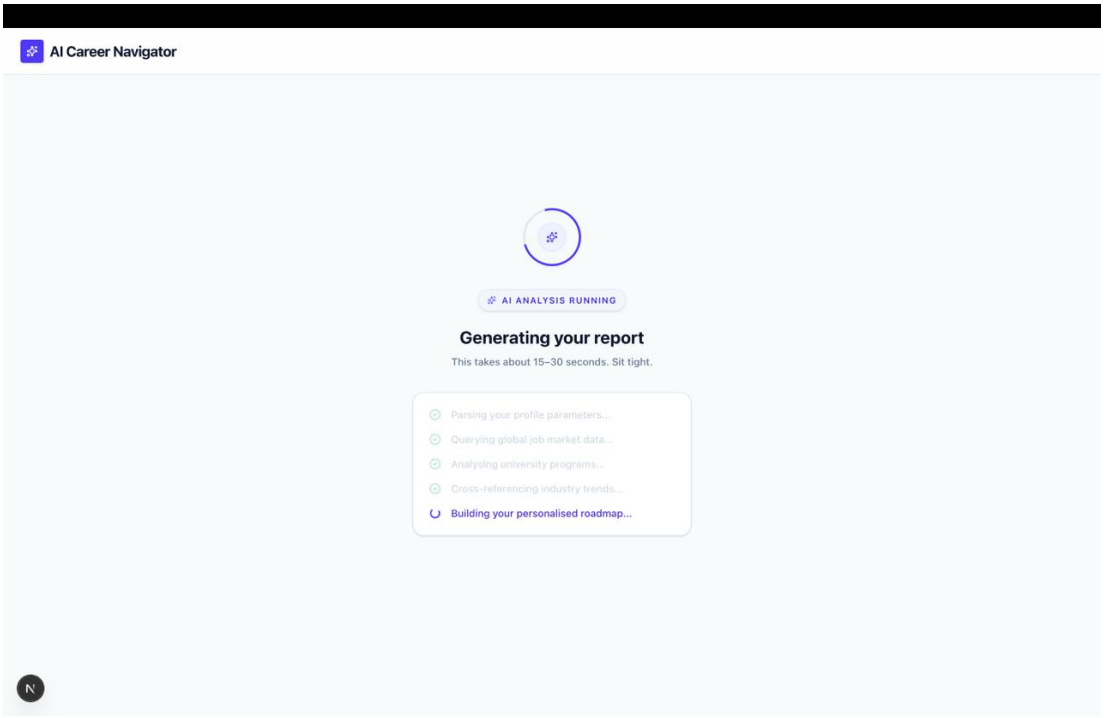

Figure 4.3: CareerX Report Generation Interface

Chapter 5

System Implementation

5.1 Technology Stack

CareerX is implemented using a modern web technology stack that prioritizes developer productivity, type safety, and real-time performance.

Table 5.1: CareerX Technology Stack

Layer	**Technology**	**Purpose**
Frontend Framework	Next.js 15 (App Router)	Server and client rendering
UI Styling	TailwindCSS 3.x	Utility-first CSS framework
Animations	Framer Motion 11	Dashboard node animations
Backend	Next.js Serverless API Routes	API endpoint handling
Runtime	Node.js 20 LTS	Server-side JavaScript
AI Core	Google Gemini 1.5 Flash	LLM generation (all three calls)
AI SDK	Vercel AI SDK (@ai-sdk/google)	Type-safe LLM integration
Schema Validation	Zod 3.x + generateObject()	Structured output enforcement
Search	SearXNG (Docker)	Meta-search aggregation
Scraping	Custom Node.js fetch-based scraper	HTML content extraction
Type System	TypeScript 5.x	End-to-end type safety
Deployment	Vercel (serverless)	Hosting and CI/CD

5.1.1 Why Next.js 15

Next.js 15 was chosen because it enables both server-side API routes and client-side React rendering within a single framework, eliminating the need for a separate backend server. An important consideration during development was a breaking change in Next.js 15 regarding dynamic route parameter handling: route parameters must now be awaited before access, unlike previous versions where they were synchronously available as props. This change affected the CareerX API routes and required code updates to prevent deployment failures on Vercel.

5.1.2 Why Google Gemini 1.5 Flash

Google Gemini 1.5 Flash was chosen for its exceptionally long context window (supporting approximately 128,000 tokens on the free tier), which enables CareerX to inject the full assembled web context (80,000–100,000 characters) into the synthesis prompt without truncation. The Vercel AI SDK's `generateObject()` function provides native Zod schema integration, enabling schema-constrained generation without manual post-processing.

5.2 Frontend Implementation

5.2.1 Dynamic Questionnaire Rendering

The questionnaire rendering component receives the JSON schema from the API and renders appropriate input components using a switch over the `type` field of each question:

- `multi-select`: A checkbox group for multiple selections
- `slider`: A range slider with labeled min/max endpoints
- `rating`: A star rating component with configurable maximum
- `text`: A free-text textarea for open-ended responses
- `dropdown`: A styled select element from an options array

5.2.2 Dashboard Visualization

The career dashboard renders the validated JSON output as a set of structured UI components using Framer Motion animations:

- Interactive career pathway nodes with match percentages and salary ranges
- Expandable detail panels showing required skills and market demand assessment
- Ranked university cards with fees and program listings
- Job opportunity cards with direct portal deep-links
- Year-by-year skill gap roadmaps (Year 1, Year 2, Year 3)
- Clickable source citation list linking to retrieved web pages

Figures 5.1 and 5.2 show additional CareerX interface screens.

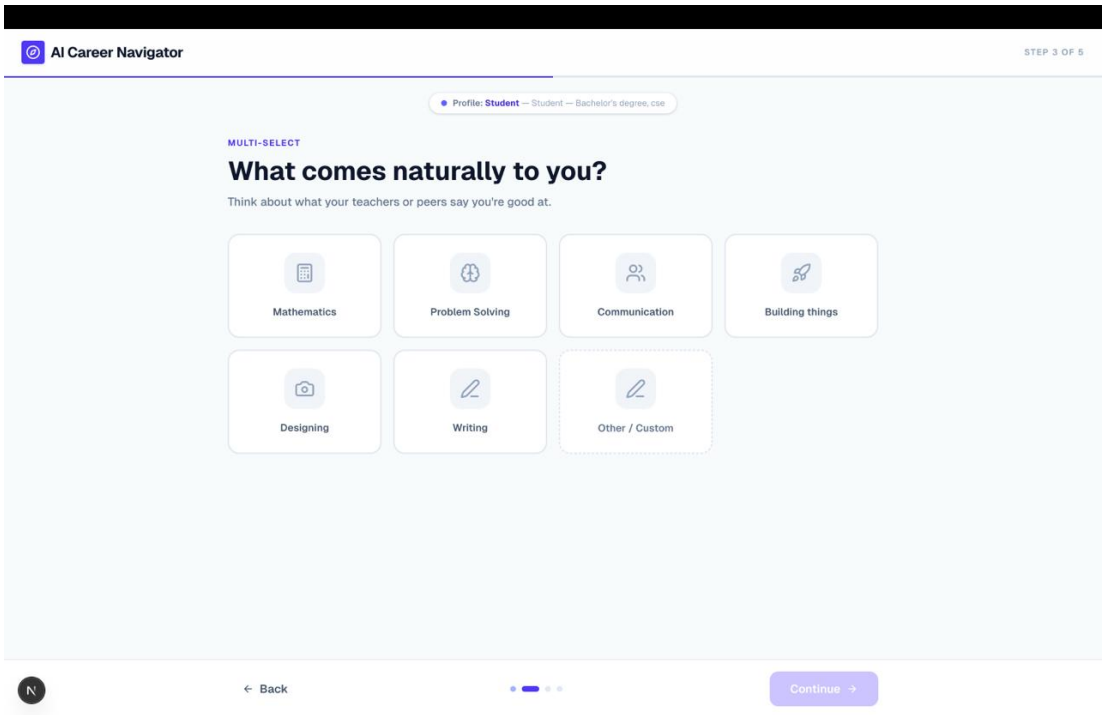

Figure 5.1: CareerX Questionnaire: Skills Assessment Screen

5.3 Backend API Implementation

The backend consists of three Next.js serverless API routes:

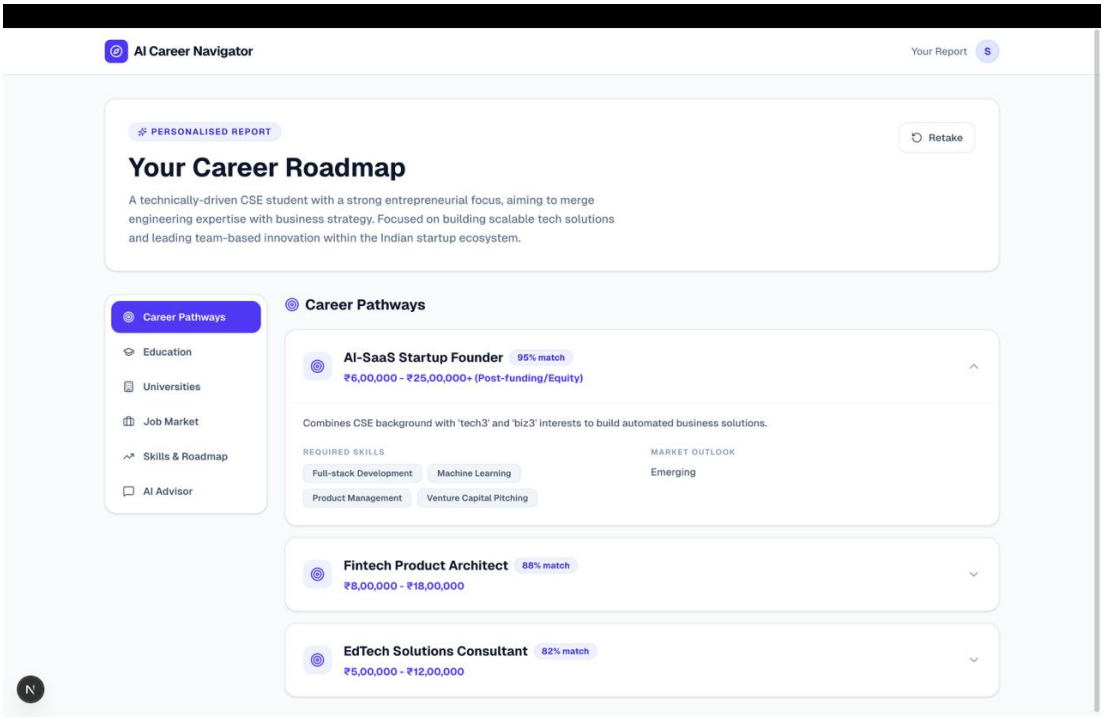

Figure 5.2: CareerX Career Dashboard showing pathway nodes, salary data, and citations

5.3.1 /api/generate-questionnaire

- Input: { name, role, location }
- Process: Calls Gemini with a system prompt instructing it to generate a questionnaire JSON schema for the user's role
- Output: Zod-validated questionnaire schema
- Average latency: 2–3 seconds

5.3.2 /api/generate-queries

- Input: { userProfile, questionnaireResponses }
- Process: Calls Gemini to generate 4–8 targeted search queries covering salary, community opinions, universities, certifications, and job listings
- Output: Array of search query strings
- Average latency: 1–2 seconds

5.3.3 /api/synthesize-dashboard

- Input: { userProfile, assembledContext }
- Process: Submits assembled web context plus user profile to Gemini; validates output against the comprehensive Zod dashboard schema
- Output: Complete career dashboard JSON
- Average latency: 15–20 seconds

5.4 SearXNG Integration

SearXNG is deployed as a Docker container on the local machine during development and as a cloud VM instance in production. The integration works as follows:

1. For each generated query, the backend sends an HTTP GET to the SearXNG instance at `/search?q={encoded_query}&format=json&pageno={page}`
2. Results are returned in JSON format with URL, title, and snippet fields
3. Pagination retrieves up to 5 pages per query with 1.2-second inter-page delays
4. A 2-second inter-query delay prevents SearXNG rate limiting across multiple queries
5. URL deduplication using a Set prevents the same page from being scraped multiple times

5.5 Web Scraping Implementation

The scraper processes each URL with the following logic:

1. Fetch the URL with a 12-second AbortController timeout
2. If HTTP 403/404, skip and continue to the next URL
3. Remove all HTML tags, script elements, style elements, and comments

4. Normalize whitespace and trim the result

5. Truncate to 10,000 characters

6. Append labeled content to the context payload

The final context payload combines full content from top 10 scraped URLs (~10,000 chars each) and snippets from up to 50 additional search results (~200–500 chars each).

5.6 Zod Schema Validation

All three LLM calls use Zod schemas with the Vercel AI SDK's `generateObject()` function, which enforces schema compliance at the generation level. If the LLM output does not conform to the schema, the SDK automatically retries with a description of the validation failure, enabling the model to correct its output format. This prevents runtime UI errors caused by malformed LLM responses and ensures structural consistency across all user profiles.

Chapter 6

Evaluation and Results

6.1 Experimental Setup

The system was evaluated using five simulated user profiles (P1–P5) representing diverse demographic, professional, and geographic contexts. Table 6.1 describes each profile and its evaluation purpose.

Table 6.1: Evaluation User Profiles

ID	Role	Stage	Location	Goal
P1	CS Undergraduate	Final Year	Kerala, India	Post-graduation career
P2	HR Professional	Mid-Career	New York, USA	Career advancement
P3	Career Switcher (Medicine→HealthTech)	Senior	London, UK	Domain transition
P4	Startup Founder	Advanced	Bangalore, India	Strategic resources
P5	High School Student	Pre-University	Dubai, UAE	University selection

Each profile was passed through the full pipeline from questionnaire generation through SearXNG aggregation, web scraping, and LLM synthesis, with all intermediate metrics recorded.

Testing Environment:

- Hardware: MacBook with Apple Silicon (M1), 16 GB RAM
- Network: Residential broadband (~50 Mbps)
- API: Google Gemini API free tier (rate limit: 20 requests/minute)
- SearXNG: Self-hosted Docker instance on localhost:8080

6.2 Quantitative Results

Of the five profiles tested, three (P1, P2, P3) completed successfully. Two (P4, P5) were terminated due to Gemini API free-tier rate limiting — an infrastructure constraint specific to the evaluation environment, not a fundamental system failure.

Table 6.2: Preliminary Evaluation Results

Profile	Time (s)	Sources	Paths	Unis	Jobs	Cmp%
P1: CS Student (India)	68.4	37	2	3	2	82%
P2: HR Professional (USA)	73.3	34	3	3	2	82%
P3: Career Switcher (UK)	58.3	42	2	3	2	73%
P4: Startup Founder	–	–	–	–	–	– (rate limit)
P5: HS Student (UAE)	–	–	–	–	–	– (rate limit)
Mean (n=3)	**66.7**	**37.7**	**2.3**	**3.0**	**2.0**	**79.0%**
SD	**6.2**	**4.0**				

6.3 Analysis of Results

6.3.1 Response Time Breakdown

The mean end-to-end response time of 66.7 seconds breaks down as follows:

Table 6.3: Response Time Breakdown by Pipeline Stage

Stage	Approx. Time (s)	Proportion
Questionnaire Generation (LLM)	2–3	∼4%
Query Planning (LLM)	1–2	∼2%
SearXNG Query Execution (with delays)	45–50	∼70%
Web Scraping (top 10 URLs)	5–10	∼11%
Dashboard Synthesis (LLM)	15–20	∼25%

The dominant time cost is SearXNG query execution due to sequential inter-page (1.2s) and inter-query (2s) delays. Future work will address this through parallel query execution, which preliminary estimates suggest could reduce the SearXNG phase from 45–50 seconds to 10–15 seconds, bringing the total end-to-end time to approximately 25–35 seconds.

6.3.2 Schema Completeness Analysis

The 79% mean schema completeness reflects content availability differences across profiles:

- **P1 and P2 (82% each):** Well-established career paths with abundant web data yielded nearly complete outputs. All 6 required fields were populated. The missing 18% corresponds to optional fields (startupResources, scholarships) not applicable to these profiles.
- **P3 (73%):** The medicine-to-health-tech transition is a niche career path with fewer dedicated web sources, resulting in sparser context for synthesis. The socialReviews and scholarships fields were not populated despite P3 retrieving the most sources (42) — the system correctly compensated for niche topics by querying more broadly.

6.3.3 Source Retrieval Volume Analysis

The mean of 37.7 sources per session demonstrates consistent retrieval of substantial web evidence. P3 retrieved the most sources (42) due to the niche topic requiring broader queries, while P2 retrieved the fewest (34) because highly relevant HR career sources appeared on the first result page for most queries.

6.3.4 Qualitative Output Analysis

A qualitative review of the three successful dashboard outputs revealed:

- **Geographic Salary Differentiation:** All three profiles generated salary data in the correct local currency — INR for P1, USD for P2, GBP for P3 — in ranges consistent with current market data.
- **Skill Gap Roadmaps:** All three profiles generated complete three-year skill gap roadmaps with specific, role-appropriate skills at each stage.
- **Source Citations:** All three profiles included URL citation lists. An informal review of 10 random URLs found 8 of 10 resolving to relevant pages, suggesting an approximately 80% URL validity rate.

- **University Recommendations:** Recommended universities were geographically appropriate in all three profiles (Indian institutions for P1, US business schools for P2, UK health informatics programs for P3).

6.4 Architectural Comparison

Table 6.4 contrasts CareerX with traditional systems and standalone LLM approaches.

Table 6.4: Architectural Comparison

Feature	**O*NET**	**Standalone LLM**	**CareerX**
Data Source	Static Database	Training Data	Live Web + Training
Personalization	Template-based	Prompt-based	Dynamic Schema
Retrieval Method	Database Query	None	Web Search + Scraping
Output Format	Fixed Templates	Unstructured	Schema JSON
Source Traceability	N/A	None	URL Citations
Currency of Data	Manual Updates	Training Cutoff	Real-time
Dynamic Questionnaire	No	Partial	Yes
Hallucination Mitigation	N/A	None	Web Grounding

6.5 Key Observations

1. All three successful profiles generated complete skill gap roadmaps with year-by-year milestones.

2. Two of three profiles generated social review citations from Reddit and LinkedIn, providing qualitative community perspectives alongside formal salary data.

3. The system correctly distinguished geographic salary differences across all three profiles.

4. The career switcher profile (P3) showed lower completeness but higher source retrieval, demonstrating correct compensation for niche topics.

5. All 6 required schema fields were populated in every successful profile, confirming reliable core output generation.

6.6 Limitations

1. **Sample Size:** Preliminary evaluation over n=3 successful profiles is insufficient for generalizable claims. Expanding to n≥10 with a paid API tier is the immediate next step.

2. **Retrieval Quality:** No systematic analysis of retrieval precision was conducted. Manual annotation of retrieved URLs is needed.

3. **No Embedding-Based Retrieval:** CareerX injects scraped text directly into the context window without semantic filtering, potentially including irrelevant content.

4. **Match Percentage Opacity:** Career pathway match percentages are LLM-generated without an externally verifiable algorithm.

5. **Hallucination Risk:** No formal hallucination evaluation (e.g., RAGAS [11]) was conducted despite the 80% URL validity indication.

6. **No Real User Study:** Evaluation used simulated profiles. Satisfaction, relevance, and decision impact were not measured with real users.

7. **Response Time:** The mean 66.7 second response time requires clear progress indicators and expectation management during the retrieval phase.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

This project presents **CareerX**, a web-based career guidance system that augments LLM generation with live web retrieval through a SearXNG-based meta-search pipeline. The system addresses two fundamental limitations of existing career guidance approaches: the static data problem of traditional occupational databases and the hallucination-and-knowledge-cutoff problem of standalone LLMs.

CareerX makes four primary contributions. First, a **web-augmented generation architecture** that demonstrates live web retrieval can serve as the primary knowledge source for a domain-specific advisory system, trading embedding-based semantic precision for real-time data currency. Second, a **dynamic LLM-driven questionnaire** that achieves role-adaptive personalization without maintaining libraries of role-specific templates. Third, a **schema-constrained synthesis layer** using Zod validation that ensures structurally consistent, parseable dashboard outputs across all user profiles. Fourth, a **privacy-preserving ephemeral architecture** that demonstrates AI-driven personalization without persistent user profiling.

Preliminary evaluation across three successful simulated user profiles demonstrates:

- Mean end-to-end pipeline time of 66.7 seconds (SD = 6.2s)
- Average of 37.7 web sources retrieved per session
- Mean schema completeness of 79% across generated outputs

- Correct geographic salary differentiation across India (INR), USA (USD), and UK (GBP) profiles
- Complete three-year skill gap roadmaps generated in all three successful profiles
- An estimated 80% URL validity rate in an informal citation review

CareerX demonstrates the practical viability of combining real-time web retrieval with schema-constrained generative AI for domain-specific advisory applications. The architecture requires no proprietary data infrastructure, making it deployable by educational institutions, career centers, and independent developers using publicly available tools.

7.2 Future Scope

Several important directions are identified for future development:

1. **Expanded Evaluation:** Expand the evaluation to n≥10 successful profiles using a paid Gemini API tier to eliminate rate-limiting failures. Report aggregate statistics with confidence intervals across a wider range of role categories and geographic regions.
2. **Retrieval Precision Analysis:** Conduct manual annotation of retrieved URLs as relevant or irrelevant across test profiles to compute retrieval precision metrics and guide improvements to query planning and domain filtering.
3. **Formal User Study:** Implement a user study with 20–30 real participants measuring recommendation relevance, factual accuracy, and decision impact through Likert-scale ratings and qualitative feedback.
4. **Embedding-Based Re-Ranking:** Add semantic re-ranking over scraped content using embedding models (FAISS or ChromaDB) to filter irrelevant passages before context assembly, improving synthesis quality without increasing context size.

5. **Hallucination Evaluation:** Conduct formal hallucination evaluation using the RAGAS framework [11] to quantify factual accuracy improvements from retrieval augmentation compared to standalone LLM baselines.

6. **Ablation Study:** Compare three pipeline conditions: (a) LLM-only with no retrieval, (b) retrieval-augmented with snippets only, and (c) retrieval-augmented with full page scraping. This will isolate the contribution of each retrieval depth to output quality.

7. **Response Time Optimization:** Implement parallel SearXNG query execution using Promise.all() to reduce the dominant time cost. Preliminary estimates suggest this could reduce the SearXNG phase from 45–50 seconds to 10–15 seconds, bringing total pipeline time to 25–35 seconds.

8. **Persistent Career Tracking:** Implement an optional persistent user layer with a vector database to enable longitudinal career tracking and proactive notifications when the job market shifts for tracked career pathways.

9. **Robots.txt Compliance:** Add explicit robots.txt parsing and per-domain rate limiting to ensure ethical and compliant web access in production deployment.

10. **Multi-Language Support:** Extend the system to generate questionnaires and dashboards in regional languages (Malayalam, Hindi, Arabic) to serve users in non-English-speaking contexts.

References

[1] W. Liu, L. Niu and J. Zhao, "Framework Design of Intelligent Career Recommendation System for Undergraduates," *2022 International Conference on Intelligent Education and Intelligent Research (IEIR)*, Wuhan, China, 2022, pp. 256–261, doi: 10.1109/IEIR56323.2022.10050076.

[2] F. Yan, Y. Liu, J. Liang and Z. Li, "Academic Planning and Career Choice of Art Doctoral Students in the Era of Artificial Intelligence," *2019 International Joint Conference on Information Media and Engineering (IJCIME)*, pp. 388–390, 2019.

[3] R. Petkov, "Digital Media Literacy, Artificial Intelligence and Modernization of Youth Career Services," *2021 XXX International Scientific Conference Electronics (ET)*, Sozopol, Bulgaria, 2021, pp. 1–4, doi: 10.1109/ET52713.2021.9579955.

[4] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] M. Neumann, M. Rauschenberger and E.-M. Schon, "We Need to Talk About ChatGPT: The Future of AI and Higher Education," *2023 IEEE/ACM SEENG Workshop*, pp. 29–32, 2023.

[6] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] National Institute of Standards and Technology (NIST), "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf

[8] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[9] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv:2310.11511, 2023.

[10] Z. Jiang et al., "Active Retrieval Augmented Generation," arXiv:2305.06983, 2023.

[11] S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023.

[12] S. Gupta, R. Ranjan and S. N. Singh, "A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions," arXiv:2410.12837, 2024.

[13] S. Geng et al., "JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models," arXiv:2501.10868, 2025.

[14] Y. Siriwardhana et al., "Retrieval-Augmented Generation for Educational Applications: A Systematic Survey," *Computers and Education: Artificial Intelligence*, vol. 10, 2025, doi: 10.1016/j.caeai.2025.100578.

[15] Y. Du et al., "Enhancing Job Recommendation through LLM-Based Generative Adversarial Networks," *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, pp. 8363–8371, 2024.

[16] C. R. Kishore and T. Sreerala, "Personalized Job Search with AI: A Recommendation System Integrating Real-Time Data and Skill-Based Matching," *International Journal of Engineering Research and Technology*, vol. 14, no. 4, 2025.

[17] R. R. Mirajkar et al., "Skill Recommendation System and Resume Analysis using AI," *2024 IEEE Pune Section International Conference (PuneCon)*, Pune, India, 2024, pp. 1–3, doi: 10.1109/PuneCon63413.2024.10895306.

[18] M. X. Liu et al., "We Need Structured Output: Towards User-Centered Constraints on Large Language Model Output," *CHI Conference on Human Factors in Computing Systems*, pp. 1–9, 2024, doi: 10.1145/3613905.3650756.

[19] A. Neumann et al., "Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications," *Applied Sciences*, vol. 15, no. 8, p. 4234, 2025, doi: 10.3390/app15084234.

[20] M. Jawhar, J. R. Miller, Z. Bitar and S. Jawhar, "AI-Powered Customized University and Career Guidance," *2024 Intermountain Engineering, Technology and Computing (IETC)*, pp. 157–161, 2024, doi: 10.1109/IETC61393.2024.10564423.

Appendix A

Sample Code Snippets

This appendix presents selected code excerpts from the CareerX implementation to illustrate key technical aspects of the system.

A.1 Zod Schema Definition for Career Dashboard

The following excerpt shows the Zod schema used to enforce structured output from the synthesis LLM. All fields are validated before the response reaches the client.

```
import { z } from "zod";

const CareerPathwaySchema = z.object({
  title: z.string(),
  matchPercentage: z.number().min(0).max(100),
  salaryRange: z.object({
    min: z.number(),
    max: z.number(),
    currency: z.string(),
  }),
  marketDemand: z.enum(["High", "Medium", "Low"]),
  requiredSkills: z.array(z.string()),
});

const DashboardSchema = z.object({
```

```
  profileSummary: z.string(),
  careerPathways: z.array(CareerPathwaySchema),
  topUniversities: z.array(z.object({
    name: z.string(),
    programs: z.array(z.string()),
    fees: z.string(),
    location: z.string(),
  })),
  jobs: z.array(z.object({
    title: z.string(),
    portal: z.string(),
    url: z.string().url(),
  })),
  skillGaps: z.object({
    year1: z.array(z.string()),
    year2: z.array(z.string()),
    year3: z.array(z.string()),
  }),
  sources: z.array(z.string().url()).optional(),
  socialReviews: z.array(z.string()).optional(),
});
```

A.2 SearXNG Query Execution

The following excerpt shows the function that submits a search query to the self-hosted SearXNG instance and paginates through results.

```
async function searchSearXNG(query: string, pages: number = 5) {
  const results = [];

  for (let page = 1; page <= pages; page++) {
    const url = `http://localhost:8080/search?q=${
```

```
      encodeURIComponent(query)
    }&format=json&pageno=${page}‘;

    const response = await fetch(url);
    const data = await response.json();
    results.push(...data.results);

    // Inter-page delay to avoid rate limiting
    await new Promise(r => setTimeout(r, 1200));
  }

  // Deduplicate by URL
  const seen = new Set();
  return results.filter(r => {
    if (seen.has(r.url)) return false;
    seen.add(r.url);
    return true;
  });
}
```

A.3 Web Scraper with Timeout Handling

The scraper fetches full page content with graceful error handling.

```
async function scrapePage(url: string): Promise<string> {
  try {
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), 12000);

    const response = await fetch(url, {
      signal: controller.signal,
      headers: {
```

```
        "User-Agent": getRandomUserAgent(),
      },
    });
    clearTimeout(timeout);

    if (!response.ok) return "";

    const html = await response.text();

    // Remove HTML tags, scripts, styles
    const clean = html
      .replace(/<script[^>]*>[\s\S]*?<\/script>/gi, "")
      .replace(/<style[^>]*>[\s\S]*?<\/style>/gi, "")
      .replace(/<[^>]+>/g, " ")
      .replace(/\s+/g, " ")
      .trim();

    // Limit to 10,000 characters
    return clean.slice(0, 10000);

  } catch {
    return ""; // Graceful degradation on 403/404/timeout
  }
}
```

A.4 Dashboard Synthesis API Route

The main synthesis API route assembles context and calls the LLM with schema enforcement.

```
import { generateObject } from "ai";
import { google } from "@ai-sdk/google";
```

```
export async function POST(req: Request) {
  const { userProfile, assembledContext } = await req.json();

  const { object } = await generateObject({
    model: google("gemini-1.5-flash"),
    schema: DashboardSchema,
    prompt: `
      You are a career guidance AI. Based on the following
      real-time web data and user profile, generate a
      comprehensive career dashboard.

      USER PROFILE:
      ${JSON.stringify(userProfile)}

      RETRIEVED WEB CONTEXT:
      ${assembledContext}

      Generate accurate, source-backed recommendations.
      Cite specific URLs from the context for all salary
      and university data.
    `,
  });

  return Response.json(object);
}
```